

PowerShell Troubleshooting Commands Guide

Essential PowerShell Cmdlets for Troubleshooting

System Information & Diagnostics

Cmdlet	Description	Example
Get-ComputerInfo	Comprehensive system information	Get-ComputerInfo -Property WindowsVersion, OsName, TotalPhysicalMemory
Get-CimInstance	CIM/WMI information retrieval	Get-CimInstance -ClassName Win32_OperatingSystem
Get-WmiObject	Legacy WMI queries (deprecated)	Get-WmiObject Win32_Processor
Get-Process	Running processes	Get-Process -Name explorer Select-Object CPU, WorkingSet
Get-Service	Service status	Get-Service -Name spooler Stop-Service -WhatIf
Get-EventLog	Windows event logs	Get-EventLog -LogName System -Newest 50
Get-WinEvent	Advanced event log filtering	Get-WinEvent -MaxEvents 10 -FilterHashtable @{LogName='Application'; Level=1; StartTime=(Get-Date).AddHours(-24)}

Network Troubleshooting

Cmdlet	Description	Example
Test-Connection	Ping/ICMP test	Test-Connection -ComputerName google.com -Count 4
Test-NetConnection	Advanced network connectivity	Test-NetConnection -ComputerName server01 -Port 3389
Resolve-DnsName	DNS resolution	Resolve-DnsName -Name google.com -Type A
Get-NetAdapter	Network adapter information	Get-NetAdapter -Name Ethernet Select-Object *
Get-NetIPConfiguration	Network configuration	Get-NetIPConfiguration -Detailed
Get-NetTCPConnection	Active TCP connections	Get-NetTCPConnection -State Listen
Get-NetRoute	Routing table	Get-NetRoute -DestinationPrefix 0.0.0.0/0

Disk, Memory & Performance

Cmdlet	Description	Example
Get-PSDrive	Available drives	Get-PSDrive -PSProvider FileSystem
Get-Volume	Volume/disk information	Get-Volume -DriveLetter C Select-Object *
Get-PhysicalDisk	Physical disk health	Get-PhysicalDisk Format-Table
Get-Counter	Performance counters	Get-Counter -Counter "\Memory\Available MBytes"
Get-Disk	Disk management	Get-Disk Where-Object OperationalStatus -eq 'Online'
Measure-Command	Measure execution time	Measure-Command { Get-ChildItem -Recurse }

Active Directory Troubleshooting (AD Module)

Cmdlet	Description	Example
Get-ADUser	User information	Get-ADUser -Identity username -Properties EmailAddress, PasswordLastSet
Get-ADGroup	Group information	Get-ADGroup -Identity "Domain Admins" -Properties Member
Get-ADComputer	Computer information	Get-ADComputer -Filter * -Properties OperatingSystem, LastLogonDate
Test-ADServiceAccount	Test service account	Test-ADServiceAccount -Identity svc-app
Get-ADDomain	Domain information	Get-ADDomain -Current LocalComputer
Get-ADReplicationFailure	Replication issues	Get-ADReplicationFailure -Target domain.local

File System & Permissions

Cmdlet	Description	Example
Get-ChildItem	List files/folders	Get-ChildItem -Path C:\Logs -Recurse -Filter *.log -ErrorAction SilentlyContinue
Get-ACL	Permissions	Get-ACL -Path C:\Windows\System32\drivers\etc\hosts
Test-Path	Check existence	Test-Path -Path C:\TEMP -PathType Container
Get-FileHash	File hashes	Get-FileHash -Path .\file.exe -Algorithm SHA256

Error Handling & Logging

Cmdlet	Description	Example
\$Error	Error variable	\$Error[0] Format-List *
\$LastExitCode	Last exit code	if (\$LASTEXITCODE -ne 0) { Write-Warning "Command failed" }
Write-Error	Custom error messages	Write-Error -Message "Could not connect to server"
Export-Csv	Export data	Get-Service Export-Csv services.csv -NoTypeInfoation

PowerShell Environment

Cmdlet	Description	Example
\$PSVersionTable	PowerShell version	\$PSVersionTable.PSVersion
Get-Module	List loaded modules	Get-Module -ListAvailable -Name ActiveDirectory
Get-Command	Available commands	Get-Command -Module ActiveDirectory Select-Object Name
Get-Help	Command documentation	Get-Help Get-Process -Detailed

Remote Management

Cmdlet	Description	Example
Enter-PSSession	Remote session	Enter-PSSession -ComputerName server01
Invoke-Command	Execute remotely	Invoke-Command -ComputerName server01 -ScriptBlock { Get-Service }
Get-WSManInstance	WS-Management	Get-WSManInstance -ResourceURI wmicimv2/Win32_Service

Common Troubleshooting Scenarios

Case: Service Failed to Start

```
powershell
# Check service status
$service = Get-Service -Name "Spooler"
$service.Status

# Check service dependencies
$service.DependentServices

# Check event log for errors
Get-WinEvent -LogName System -MaxEvents 10 | Where-Object {$_.ProviderName -like "*Service*"}

# Try to start with logging
```

```
Start-Service -Name "Spooler" -Verbose -ErrorAction SilentlyContinue
if ($?) { Write-Host "Service started successfully" } else { Write-Host
"Error: $($Error[0])" }
```

Case: Network Connectivity Issues

powershell

```
# Combined network test
$hostname = "google.com"
Test-Connection -ComputerName $hostname -Count 2 -Quiet
Test-NetConnection -ComputerName $hostname -Port 443
Resolve-DnsName -Name $hostname

# Get network configuration
Get-NetIPConfiguration | Select-Object InterfaceAlias, IPv4Address, DNSServer
```

Case: Disk Space Issues

powershell

```
# Check disk space
Get-PSDrive -PSProvider FileSystem | Where-Object {$_.Free -gt 0} |
    Select-Object Name, @{N='Used (GB)';E={[math]::Round(($_.Used/1GB),2)}},
    @{N='Free (GB)';E={[math]::Round(($_.Free/1GB),2)}}

# Find large files
Get-ChildItem -Path C:\ -File -Recurse -ErrorAction SilentlyContinue |
    Sort-Object Length -Descending |
    Select-Object -First 10
```

Case: User Access Issues

powershell

```
# Check user permissions
$user = "username"
$file = "C:\Users\username\Documents\file.txt"
Get-ACL -Path $file |
    Select-Object -ExpandProperty Access |
    Where-Object IdentityReference -like "*$user*"

# Get user group memberships
Get-ADUser -Identity $user -Properties MemberOf |
    Select-Object -ExpandProperty MemberOf |
    ForEach-Object { (Get-ADGroup $_).Name }
```

Quick Troubleshooting One-Liners

powershell

```
# Check system uptime
(Get-CimInstance Win32_OperatingSystem).LastBootUpTime

# Find stopped services
Get-Service | Where-Object {$_.Status -eq 'Stopped'}
```

```
# Check for recent critical events
Get-WinEvent -MaxEvents 20 -FilterHashtable @{Level=1; LogName='System'}

# Verify network connectivity to critical servers
"server01","server02","dc01","gateway" | ForEach-Object { Test-Connection $_
-Count 1 -Quiet }

# Check running processes by CPU usage
Get-Process | Sort-Object CPU -Descending | Select-Object -First 10

# Find recently modified files in a directory
Get-ChildItem -Path C:\ -File -Recurse -ErrorAction SilentlyContinue | Where-
Object {$_ .LastWriteTime -gt (Get-Date).AddHours(-2)}
```

Useful PowerShell Variables

Variable	Description
\$_	Current object in pipeline
\$true / \$false	Boolean values
\$null	Null/empty value
\$host	Current host information
\$input	Pipeline input
\$PID	Process ID of current session
\$env:	Environment variables (e.g., \$env:COMPUTERNAME)

Error Handling Patterns

powershell

```
# Try-Catch-Finally pattern
try {
    Get-Service -Name "NonExistentService" -ErrorAction Stop
}
catch {
    Write-Error "Error occurred: $($_.Exception.Message)"
}
finally {
    Write-Host "Operation completed"
}

# Using -ErrorVariable
Get-Process -Name "nonexistent*" -ErrorVariable myErrors
$myErrors

# Silent error handling with manual check
$result = Get-Service -Name "Spooler" -ErrorAction SilentlyContinue
if ($null -eq $result) {
    Write-Warning "Service not found"
}
```